STUDY SWAP

PROJECT VIVA STUDY GUIDE & BLUEPRINT

Prepared for Course Viva

Project Name:	StudySwap Marketplace
Course:	Mobile Application Development (MAD)
Semester:	6th Semester / Bachelor of Computer Science
Technology Stack:	Flutter (Dart), Firebase Auth, Cloud Firestore, Firebase Storage
Prepared By:	Rehan Khan
Academic Session:	2026
Document Scope:	Architecture, CRUD Operations, workflows, and Teacher-Level Q/A

Note: This study guide contains actual extraction and analysis of the active codebase.

Do not share outside of immediate StudySwap course review teams.

PART 1: PROJECT OVERVIEW

1. Project Purpose

StudySwap is a specialized peer-to-peer student marketplace designed specifically for university campuses. The primary objective is to enable college students to easily buy, sell, exchange, or share academic resources such as textbooks, hand-written lecture notes, scientific calculators, stationery, and lab equipment. Rather than relying on commercial, generic e-commerce platforms, StudySwap provides a localized, secure, and student-focused resource pool. It solves key problems like high textbook costs, wasteful disposal of active academic resources, and barrier-to-entry peer communication on campus.

2. Dynamic Buyer Workflow

The Buyer's flow in StudySwap proceeds through a structured sequence of screens:

1. Entry & Splash: The app launches on the splash screen, checks if the user is authenticated, and directs them to the Login/Register flow, or directly to the Home Screen if already signed in.
2. Home Screen: Once authenticated, a Buyer lands on a dashboard containing high-quality banner slides and categories.
3. Category Filtration & Search: Buyers filter listings by categories (Books, Notes, etc.) or search for items using the Search bar.
4. Item Detail Interaction: Selecting an item opens the product details screen. Buyers can see the item title, price, seller's name, stock level, condition review, and description.
5. Cart & Wishlist: Buyers can toggle the "Favorite" button to persist items in their Wishlist. They can also tap "Add to Cart" or "Buy Now" to queue the item.
6. Real-Time Chatting: By clicking "Chat with Seller", a buyer opens a direct real-time chat screen relative to that specific listing, allowing negotiations.
7. Checkout: Proceeding to the Cart screen, the buyer confirms quantities and clicks "Checkout". This runs a secure Firestore batch transaction to create orders, decrement stock, and clear the cart.
8. Order Tracking: Tapping standard orders in the profile list displays active orders with update states.

3. Dynamic Seller Workflow

The Seller's flow in StudySwap proceeds through these operational phases:

1. Account Selection: Users select "Seller" during registration or login, storing `accountType: "Seller"` in their Firestore document.
2. Main Navigation: Sellers access the standard search-and-browse page. They can open the side Drawer and tap "Dashboard" to slide open the Seller Hub as a back-navigable subpage route.
3. Seller Dashboard: Displays key analytics like Total Revenue, Monthly/Today sales, Active Listings, Items Sold, Low/Out of Stock counters, and a table of the 5 latest sales.
4. Listing Creation & Storage: Tapping "List New Item" allows the seller to fill title, price, stock, category, condition, and description. Sellers can select mock assets or upload live photos using the FilePicker, which uploads the image file to Firebase Storage.
5. Listing Maintenance: Sellers can browse their listings, tap to edit stock/details, or swipe to delete, which

automatically calls Firestore delete and deletes assets from Storage.

6. Order Fulfillment Updates: When an order is received, the seller sees it on their dashboard. They can change the order status (Pending -> Shipped -> Delivered -> Cancelled) via a dropdown, which automatically logs the completion timestamp and alerts the buyer.

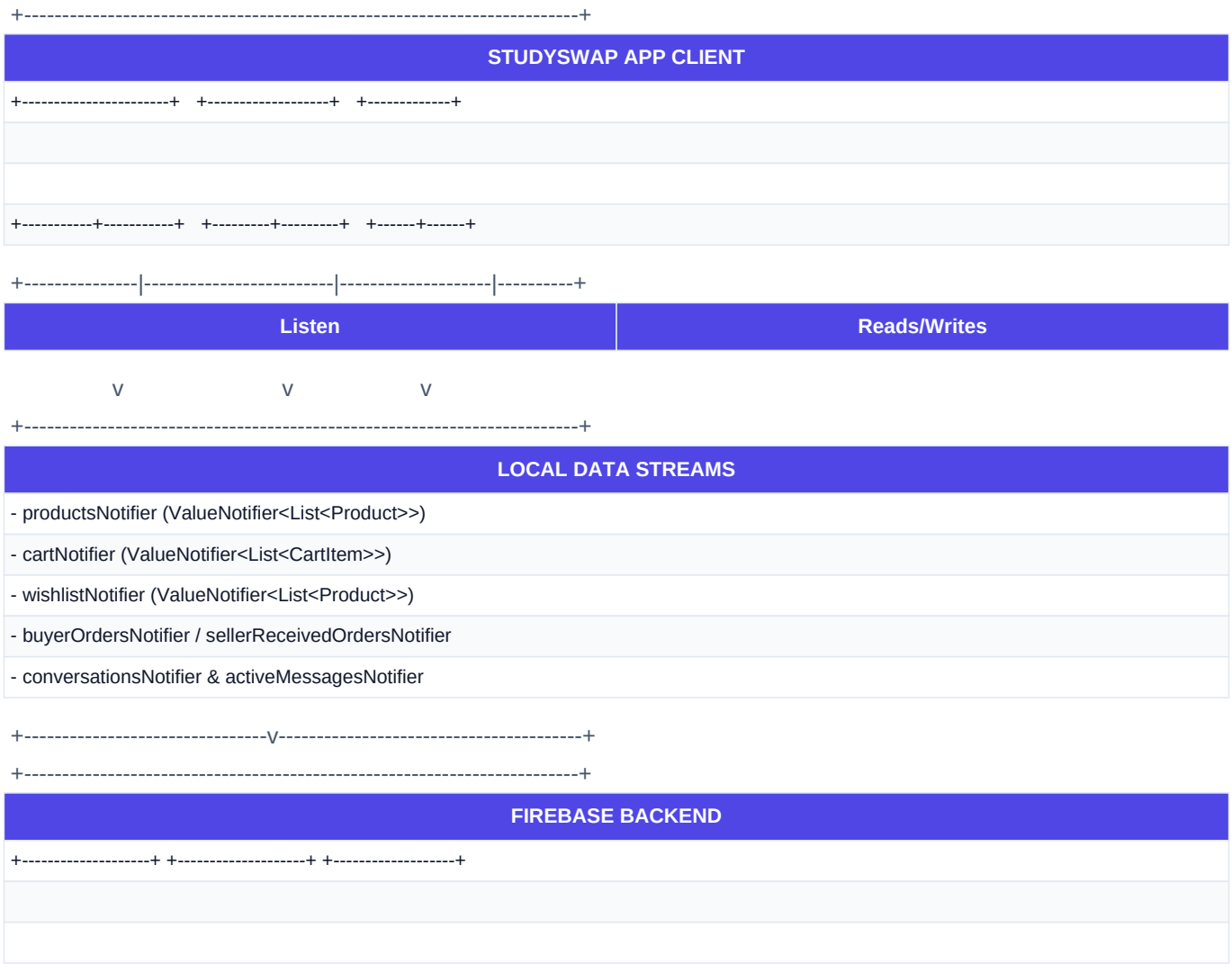
4. Firebase Services Architecture

StudySwap relies completely on Firebase for its serverless backend:

1. Firebase Authentication: Handles signup, secure password validation, persistent login sessions, and role detection.
2. Cloud Firestore: A NoSQL document database used to store users, listings, cart items, wishlist saves, order logs, messages, and notifications in real-time.
3. Firebase Storage: Stores binary user artifacts, specifically selected listing images and user profile pictures.
4. Real-time Streams: Listeners hook directly to Firestore collections via ValueNotifiers, prompting UI elements to update instantly without manual refreshes.

5. Architectural Blueprint Diagram

The following blueprint demonstrates how StudySwap aligns its UI layer with the Firebase backend:



```
+--------------------+ +--------------------+ +--------------------+

 +-------------------------------------------------------------------------+
```

PART 2: FIREBASE BACKEND DETAILS

1. Firebase Initialization

StudySwap references and launches Firebase connection settings using `main.dart`. Upon launching the app, the system calls:

```
WidgetsFlutterBinding.ensureInitialized();
await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
);
AppState.init();
```

`DefaultFirebaseOptions.currentPlatform` extracts configurations for Android or iOS platforms (e.g. database URLs, API Keys, Sender IDs) stored inside `lib/firebase_options.dart`.

2. Firebase Authentication

- Purpose: Provides credentials management, secure authorization tokens, password verification, and authentication state events.
- Usage: Every profile interaction in StudySwap uses it.
- Files: `lib/screens/login_screen.dart`, `lib/screens/register_screen.dart`, `lib/screens/settings_screen.dart`, `lib/state/app_state.dart`.
- Operations:
- Profile registration: `FirebaseAuth.instance.createUserWithEmailAndPassword(email, password)`
- Profile Login: `FirebaseAuth.instance.signInWithEmailAndPassword(email, password)`
- Active User state listener: `_auth.authStateChanges()` hook in `AppState.init()`.

3. Cloud Firestore NoSQL Database

- Purpose: A distributed JSON-document storage structure. We use it to record our relational entities without complex SQL joins.
- Usage: Holds active catalogs, buyer orders, cart indexes, messaging channels, and notification logs.
- Files: `lib/state/app_state.dart`, `lib/screens/analytics_screen.dart`, `lib/screens/dashboard_screen.dart`, and all transaction pages.
- Structure: Collections hold Documents. Documents hold key-value maps. We use references (`doc()`, `collection()`) to query data.

4. Firebase Storage

- Purpose: Server binary storage for high-quality static assets.
- Usage: Selected photos from student resources or profile portraits are held as raw files and assigned unique access URLs.
- Files: `lib/state/app_state.dart` (specifically `_uploadProductImage` and `uploadAndUpdateProfileImage`).
- Operations:

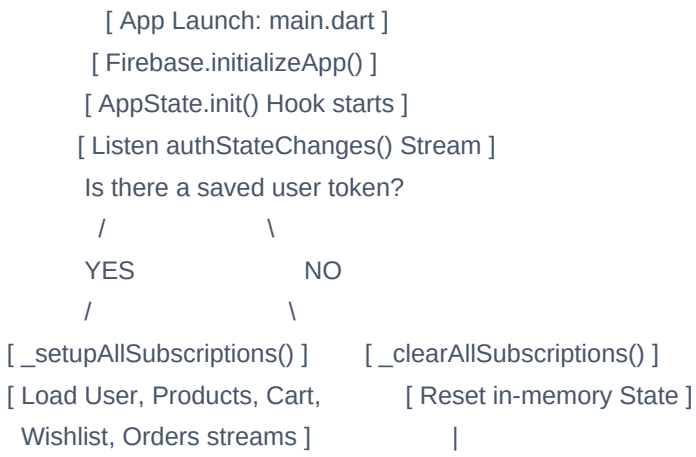
- Fetch bucket references: `FirebaseStorage.instance.ref()`.
- Upload file raw bytes: `storageRef.putFile()` or `storageRef.putData()`.
- Obtain public link: `snapshot.ref.getDownloadURL()`.

5. Realtime Streams & Listeners

- Purpose: Keep the client UI dynamically in sync with the cloud database.
- Logic: Firestore pushes data changes automatically to client devices using underlying web sockets. In `app_state.dart`, we subscribe to queries using `.snapshots()`. Every time a document matches the query or updates, the stream fires, yielding `QuerySnapshot` or `DocumentSnapshot`. We pass this data to our Flutter `ValueNotifiers`, causing listeners block to rebuild the UI instantly.

6. Authentication State Flow chart

The diagram below describes the session routing lifecycle during initialization:



PART 3: FIRESTORE DATABASE DESIGN

1. Physical Collections & NoSQL Hierarchy

StudySwap contains 7 primary collections inside Cloud Firestore. The collections represent decoupled logical entities structured as follows:

Collection ID	Document ID Scheme	Database Relationships	Primary Screen(s) utilizing it
users	Firebase Auth User ...	One-to-One with Aut...	profile_screen, log...
products	Auto-Timestamp Stri...	One-to-Many via sel...	sell_screen, detail...
orders	Auto-generated ID	Many-to-One via buy...	orders_screen, dash...
savedItems	`${uid}_${productId...	Junction table user...	wishlist_screen, de...
cart	`${uid}_${productId...	Junction table buye...	cart_screen, detail...
conversations	Auto-generated ID	Array participants:...	messages_screen, ch...
notifications	Auto-generated ID	Many-to-One via use...	notifications_scree...

---

2. Collection Schema Details

A. Collection: `users`

- Purpose: Stores profiles, name details, contact numbers, and access rights.
- Document ID: The user's Firebase Auth `UID`.
- Schema Fields:
- `uid` (String): Unique identifier.
- `fullName` (String): Display name of student.
- `email` (String): CUI academic address.
- `phone` (String): Cell number for exchange coordinates.
- `accountType` (String): User role, either `'Buyer'` or `'Seller'`.
- `profileImage` (String): URL of the uploaded image in Storage.
- `createdAt` (Timestamp): Timestamp when profile was built.

B. Collection: `products`

- Purpose: Active catalog listing of study resources on the market.
- Document ID: Unique epoch timestamp milliseconds string (e.g. `1784827419`).
- Schema Fields:
- `title` (String): Product title.
- `price` (Double): Selling price.
- `imageUrl` (String): HTTP link to image file/mock resource.
- `category` (String): Academic filter (e.g., `'Books'`).

- `condition` (String): Usage state indicator (e.g., `'Like New'`).
- `description` (String): Listing details notes.
- `sellerName` (String): Display name of listing author.
- `sellerId` (String): UID of listing owner.
- `rating` (Double): Default 4.5.
- `reviewsCount` (Int): Default 12.
- `isSold` (Boolean): Flag defining if item is sold.
- `stock` (Int): Active stock quantity (>= 0).
- `status` (String): Status code of item (`'active'` or `'inactive'`).
- `createdAt` (Timestamp): Server timestamp of creation.

C. Collection: `orders`

- Purpose: Tracks order history, quantities, completion, and financial summaries.
- Document ID: Auto-generated string.
- Schema Fields:
- `buyerId` (String): UID of the purchasing student.
- `buyerName` (String): Name of the buyer.
- `sellerId` (String): UID of listing owner.
- `sellerName` (String): Name of the seller.
- `productId` (String): ID of the product.
- `price` (Double): Price of the transaction.
- `quantity` (Int): Quantity of items purchased.
- `status` (String): Fulfillment status (`'Pending'`, `'Shipped'`, `'Delivered'`, `'Cancelled'`).
- `purchaseDate` (Timestamp): Order date.
- `createdAt` (Timestamp): Creation server timestamp.
- `completedAt` (Timestamp): Delivery completion server timestamp.
- `product` (Nested Map): Snapshot of product data `{id, title, imageUrl, category, condition, description}`.

D. Collection: `savedItems` (Wishlist)

- Purpose: Serves as a junction collection matching users to their saved listings.
- Document ID: `${uid}_${productId}` (Ensures a user saves a listing only once. Tabbing favorite deletes the doc or creates it).
- Schema Fields:
- `userId` (String): UID of interest participant.
- `productId` (String): ID of liked product.
- `savedAt` (Timestamp): Timestamp of click.

E. Collection: `cart`

- Purpose: Holds active item queues for target buyers before checkouts.
- Document ID: `${uid}_${productId}` (Combines buyer ID and product ID, ensuring quantity increments rather than multiple docs).
- Schema Fields:
- `buyerId` (String): Target purchaser UID.
- `productId` (String): ID of product.

- `quantity` (Int): Total item counts to buy.
- `productTitle` / `productPrice` / `productImageUrl` (Strings): Sharded product metadata fields.
- `productSellerId` (String): UID of seller.

F. Collection: `conversations`

- Purpose: Tracks dialog channels between unique buyer/seller pairs on listings.
- Document ID: Auto-generated string.
- Schema Fields:
- `participants` (Array of Strings): Holds `[buyerId, sellerId]`.
- `buyerId` (String): Buyer UID.
- `sellerId` (String): Seller UID.
- `buyerName` / `sellerName` (String): Display names.
- `productId` (String): Reference item.
- `productTitle` / `productImageUrl` (String): Reference details.
- `lastMessage` (String): Preview snippet of chat.
- `lastMessageTime` (Timestamp): Timestamp of last active sentence.
- `unreadCount` (Map): Map containing `{buyerId: count, sellerId: count}`.

G. Collection: `notifications`

- Purpose: Serves system, messaging, and sales alerts.
- Document ID: Auto-generated ID.
- Schema Fields:
- `notificationId` (String): Self document ID.
- `userId` (String): Target profile UID.
- `title` (String): Bold header description.
- `body` (String): Details message.
- `type` (String): `'Messages'`, `'Sales'`, or `'System'`.
- `relatedId` (String): Optional target reference document ID.
- `isRead` (Boolean): Boolean flag for notification read status.
- `timestamp` (Timestamp): Creation server timestamp.

PART 4: EXPLAIN EVERY IMPORTANT CRUD OPERA

In StudySwap, Firestore reads and writes are executed via the official `cloud_firestore` package. Below is the technical breakdown of the 15 important operations utilized in StudySwap.

1. doc() and collection()

- Code Snippet:

```
_firestore.collection('savedItems').doc("${uid}_${product.id}")
```

- Explanation: `collection('col_name')` creates or fetches a reference to a parent folder (logical table) in Firestore. `doc('doc_id')` points to a specific document in that collection.
- Rationale: Used to point to a specific record. In our code, we do this to create or query composite keys like `${uid}_${product.id}` for carts and wishlists.
- Process: Runs locally. Firebase does not execute a network call until you run a read/write operation like `get()`, `set()`, or `update()`.

---

2. set()

- Code Snippet:

```
await _firestore.collection('users').doc(user.uid).set({
  'uid': user.uid,
  'fullName': _nameController.text.trim(),
  'email': _emailController.text.trim(),
  'phone': _phoneController.text.trim(),
  'accountType': accountType,
  'profileImage': '',
  'createdAt': FieldValue.serverTimestamp(),
});
```

- Explanation: `set()` writes a document to Firestore at the specified location. If the document already exists, `set()` completely overwrites its content unless you pass the option `SetOptions(merge: true)`.
- Rationale: Renders profile creations after auth signups, ensuring that the Firestore document sits exactly under the user's Auth `UID`.
- Difference: Unlike `add()`, which auto-generates a random ID, `set()` allows the client to define the target document ID.

---

3. add()

- Code Snippet: There is no native `.add()` block in app_state.dart because we use `doc().set()` or `batch.set()` to enforce custom/deterministic IDs, or `doc()` without arguments to generate unique IDs before writing.
- Example of doc().set() representing the exact equivalent of add():

```
final docRef = _firestore.collection('notifications').doc();
await docRef.set({ 'notificationId': docRef.id, ... });
```

- Explanation: In Firestore, `.add()` is a shorthand helper. Calling `collection.add(data)` is equivalent to calling `collection.doc().set(data)`.
- Difference: With `.add(data)`, you cannot get the document ID before writing. With `.doc().set(data)`, you generate the ID first (`docRef.id`), store that ID inside the document itself (as `notificationId`), and then write.

---

4. update()

- Code Snippet:

```
await _firestore.collection('orders').doc(orderId).update(updates);
```

- Explanation: `update()` modifies specific key-value fields inside an existing document. It does not overwrite the entire document.
- Rationale: Modifies states like changing an order status (Pending -> Delivered) or adding 1 to cart quantities without erasing secondary fields.
- Difference: `update()` fails if the document does not exist, whereas `set()` creates a new document.

---

5. delete()

- Code Snippet:

```
await _firestore.collection('cart').doc("${uid}_${product.id}").delete();
```

- Explanation: `delete()` completely removes the document at the target document reference path.
- Rationale: Executed when remove items from cart or wishlist, and when deleting listings.

---

6. get()

- Code Snippet:

```
final docSnapshot = await docRef.get();
if (docSnapshot.exists) { ... }
```

- Explanation: `get()` carries out a single, one-time read request to the database. It returns a future with a snapshot of the document's fields at that moment.
- Rationale: Checks if a product is already in the wishlist, or retrieves a profile document to inspect account roles.
- Difference: `get()` is a one-time read operation, whereas `snapshots()` sets up a live subscription.

---

7. snapshots()

- Code Snippet:

```
_firestore.collection('products').snapshots().listen((prodSnapshot) { ... });
```

- Explanation: `snapshots()` creates a live data stream of query results.
- Rationale: Kept active permanently in `AppState` subscriptions, ensuring listings, cart numbers, notifications, and

orders update in real-time.
- Difference: `snapshots()` keeps a persistent connection open, whereas `get()` is a one-time read.

---

8. where()

- Code Snippet:

```
.collection('savedItems').where('userId', isEqualTo: uid)
```

- Explanation: `where()` filters document outputs from a collection based on specific field values.
- Rationale: Restricts cart item streams to the currently logged-in user (`buyerId == uid`).

---

9. orderBy()

- Code Snippet:

```
.collection('messages').orderBy('timestamp', descending: false)
```

- Explanation: `orderBy()` sorts query results by a specified field in ascending or descending order.
- Rationale: Ensures chat messages render in correct chronological order in the chat screen.

---

10. limit()

- Code Snippet:

```
_firestore.collection('products').limit(1).get()
```

- Explanation: `limit()` caps the number of matching documents returned by a query.
- Rationale: Optimizes reads when checking if a collection is empty, or checking if a conversation between two users already exists.

---

11. toMap() and fromFirestore()

- Code Snippet:

```
factory Product.fromFirestore(Map<String, dynamic> data, String id) { ... }
Map<String, dynamic> toMap() { ... }
```

- Explanation: Helpers that paint translation boundaries between strongly-typed Dart objects (`Product`) and maps (`Map<String, dynamic>`) stored in Firestore.
- Rationale: Dart requires type safety, while Firestore stores documents as JSON-like documents.

---

12. FieldValue.serverTimestamp()

- Code Snippet:

```
'createdAt': FieldValue.serverTimestamp()
```

- Explanation: A token passed to Firestore that tells the server to insert its current clock time.
- Rationale: Solves client time drift issues and ensures consistent timestamps for orders and notifications.

---

13. FieldValue.increment()

- Code Snippet:

```
'unreadCount.$otherUid': FieldValue.increment(1)
```

- Explanation: A mathematical token that tells the Firestore server to increment a numeric field by a specified value.
- Rationale: Used in `addMessage` to increment the recipient's unread message counter.
- Key Advantage: Avoids write conflicts (race conditions) that happen when multiple clients read, modify, and write a counter simultaneously.

PART 5: AUTHENTICATION FLOW

Authentication in StudySwap coordinates user access by wrapping Firebase Authentication and Firestore together in a synchronized lifecycle.

1. Account Creation (Registration)

When a student fills out the registration form in `register_screen.dart` and clicks "Create Profile", the following steps occur:

1. Input Verification: Client-side validation checks requirements for full name, academic email format (`@*.edu` checks etc.), phone number length, password strength, and matching passwords.
2. Firebase account creation: If clean, the app triggers:

`await FirebaseAuth.instance.createUserWithEmailAndPassword(email, password)`

This registers the user with Firebase Authentication using an encrypted password.

3. Creation of Firestore Document: Once the above returns a credentials object, the user's Auth `UID` is extracted. The code immediately registers details in the database:

`await FirebaseFirestore.instance.collection('users').doc(user.uid).set({ ... })`

This stores the profile's non-credentials details (`uid`, `fullName`, `email`, `phone`, `accountType: "Buyer" | "Seller"`).

4. System Notification: The method uploads a default system welcome alert.
5. Home transition: The app uses `Navigator.pushAndRemoveUntil` to navigate to `HomeScreen()`.

---

2. Login Flow

When a user inputs their email and password in `login_screen.dart` and taps "Sign In":

1. Input check: Validates standard inputs.
2. Firebase Authentication:

`await FirebaseAuth.instance.signInWithEmailAndPassword(email, password)`

This requests credentials validation from the Firebase server.

3. Account Type Parsing: If the login succeeds, the screen fetches the user's Firestore document manually:

`final userDoc = await FirebaseFirestore.instance.collection('users').doc(user.uid).get();`

Reads the `accountType` field.

4. Set AppState: Updates the state manager's user role value notifier:

`AppState.userRoleNotifier.value = userDoc.data()?['accountType'] ?? 'Buyer';`

5. Home transition: Navigates to `HomeScreen()`.

---

3. Logout Flow

Tapping the logout list item in `profile_screen.dart` triggers:

`await AppState.signOut();`

Inside `AppState.signOut()`:

1. All Firestore listeners are destroyed to prevent memory leaks and unauthorized access warnings: `_userSubscription?.cancel(); _productsSubscription?.cancel(); ...`
2. The user's local variables (name, roles, list states) are reset to empty values.
3. Signs out of Firebase Authentication: `await _auth.signOut();`
4. The UI routes back to `LoginScreen()`.

---

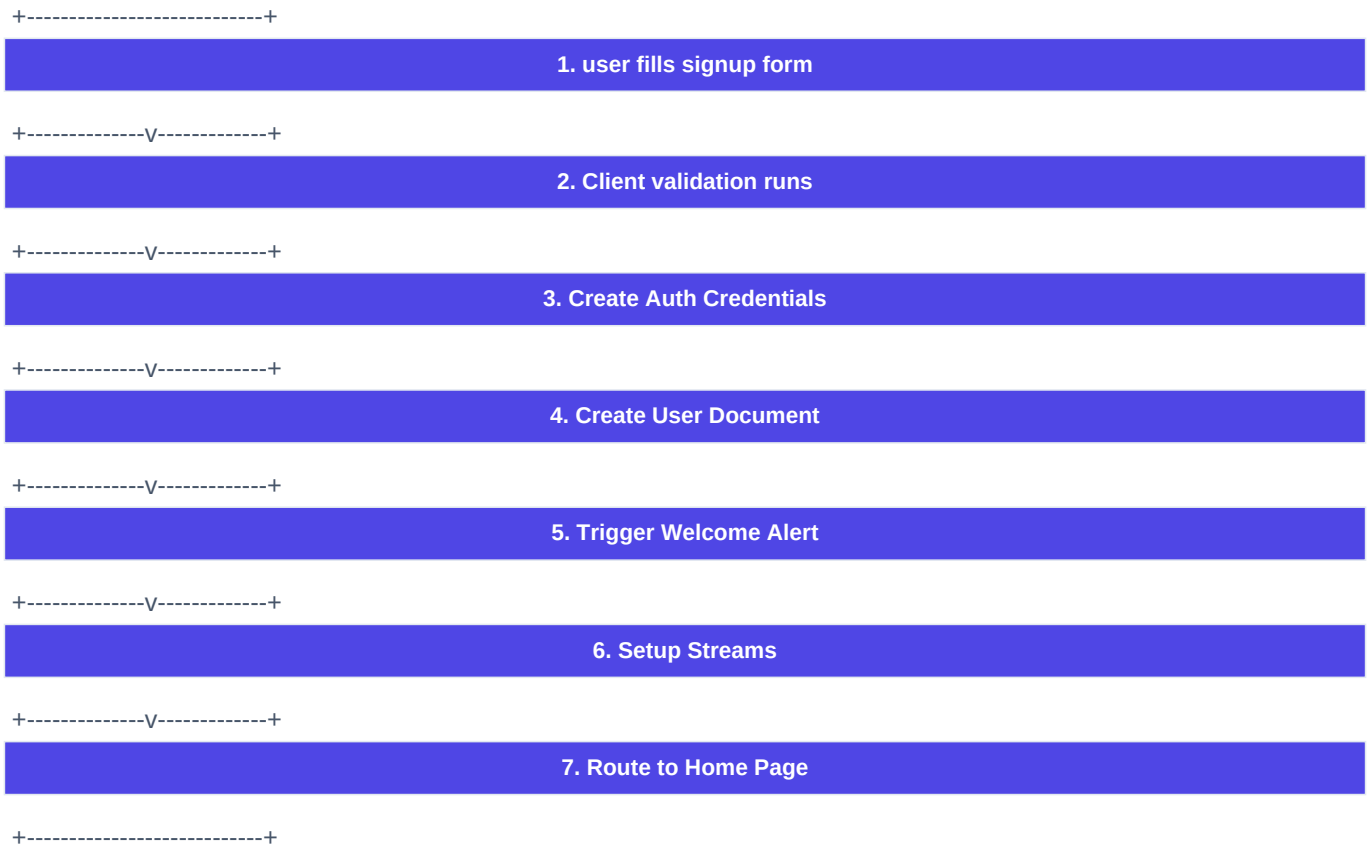
4. Role Detection and Separation

- Database Driven: A user's role is stored in Firestore under the `accountType` field in the `users` collection.
- Local State Hook: When a user authenticates, `AppState._setupAllSubscriptions(uid)` sets up a live listener on the user's doc. If the role changes in the database, `AppState.userRoleNotifier` updates automatically.
- Front-End Rendering:
- Buyers see a standard bottom navigation bar with options for Home, Wishlist, Cart, Messages, and Profile.
- Sellers see the exact same home browsing screen as buyers (so they can search resources). However, their side drawer includes a "Dashboard" item. Tapping it opens the `SellerDashboardScreen` subpage.

---

5. Register-to-Home Step-by-Step Flow Chart

The checklist below describes the sequential operations during registration:



PART 6: SELLER WORKFLOW & OPERATIONS

This chapter outlines the logical and user-interactive sequence of events that characterize the study guide's Seller operations in StudySwap.

1. Seller Login & Dashboard Routing

- Login: Sellers input credentials in `login_screen.dart`. Upon success, the user role value notifier configures as `'Seller'`.
- Routing: The app shell renders the standard purchase list tabs. Access to the dashboard is routed through the side navigation Drawer. Selecting "Dashboard" pops the drawer and pushes `SellerDashboardScreen` as a subpage route.

---

2. listing/Uploading a Product

Tapping the "List New Item" quick action button in the Dashboard navigates to `SellScreen`. The listing process flows as follows:

1. Sourcing Visuals: Sellers can select a default category mock vector image or choose a custom file from their local gallery.
2. Form Validation: Sellers fill out the title, price, stock quantity, category description, condition chip, and description.
3. Binary Upload to Storage (Pick & Upload):

- Selecting a custom image triggers `FilePicker.pickFiles()`.
- File picker returns a local filepath (or bytes on web).
- When the user clicks "Publish Listing", `AppState.addProduct()` is called.
- If the image path is local (not starting with http/assets), the app uploads it to Firebase Storage:

`FirebaseStorage.instance.ref().child('product_images/$productId')`

using `.putFile()` (mobile) or `.putData()` (web).

- The app waits (`await`) for the upload task to finish and retrieves the download URL using `getDownloadURL()`.

4. Writing Product to Firestore:

`await _firestore.collection('products').doc(product.id).set({...})`

This stores the retrieved download URL inside the document under the `imageUrl` key.

5. Notification Trigger: Stores an alert document in the `notifications` collection to inform the seller.

---

3. Editing and Deleting Listings

- Editing listings: Selecting a product from the Catalog tab or My Listings screen navigates to `EditListingScreen`. Changes are committed using `update()` on the target document:

`await _firestore.collection('products').doc(product.id).update(data);`

If a new custom image is picked to replace the old one, the new file is uploaded to Storage first.

- Deleting listings: Swiping to delete or tapping the delete bin icon triggers `deleteProduct(id)`. This:
- Calls `storageRef.delete()` to remove the image file.
- Calls `_firestore.collection('products').doc(id).delete()`.

---

4. Automatic Stock and Inventory Updates

- Order checkout decrement: During a buyer's checkout, a transaction-like batch commits. For every item in the cart, the system checks the catalog stock level. It calculates:

`newStock = latestProd.stock - item.quantity`

- Batch update: The batch updates the product document's stock field. If stock drops to 0, `isSold` is set to `true`, and the product is marked as `'inactive'`:

`batch.update(prodRef, { 'stock': 0, 'isSold': true });`

- Automated System alerts:
- If stock drops between 1 and 5, low stock alerts are sent to the seller.
- If stock hits 0, out of stock alerts are sent.

---

5. Orders Received Operations

Incoming orders are displayed in a real-time list on the dashboard under "Recent Sales". Sellers manage orders as follows:

- Order Status Transitions: Sellers open the order management dialog or screen, and change the status using a dropdown.
- Modifying Firestore: Tapping the state calls:

`await _firestore.collection('orders').doc(orderId).update({'status': newStatus});`

- Real-time completed checks: If the seller marks the status as `'Delivered'`, the system automatically appends a completion timestamp:

`updates['completedAt'] = FieldValue.serverTimestamp();`

This triggers calculations for total revenue, graphs, and completed profiles.

---

6. Financial Analytics Dashboard & Trend Painting

- Metrics panel: The dashboard calculates metrics from the active orders stream:
- Total Revenue: Sum of (price * quantity) of all orders with status `'Delivered'`.
- Active listings count: Count of items where `status == 'active'` and `sellerId == uid`.
- Items sold: Sum of quantities of delivered orders.
- Interactive custom Trend Chart: In `AnalyticsScreen`, changing filters (Today, 7 days, 30 days) reroutes calculations.
- Line curve (Bezier path): Renders revenue trends plotted across completed datetimes.
- Bar charts: Render orders volume trends.
- Class structure: `FlexibleChartPainter` handles custom canvas lines, grids, ticks, and dots matching the premium

theme colors.

- Customer cohort profiling: Tracks buyer counts from orders:
- Total buyers: Count of unique buyer IDs.
- Returning buyers: Count of buyer IDs with more than one completed order.
- New buyers: Buyer IDs with exactly one completed order.

PART 7: BUYER WORKFLOW & LOGICAL SEQUENC

This chapter details the sequential operations performed by a Buyer in StudySwap.

1. Account Creation and Login

Buyers select "Buyer" during registration. This writes `accountType: "Buyer"` to Firestore. When they log in, the screen manually validates their credentials, queries their account type, updates `userRoleNotifier`, and directs them to the home screen.

---

2. Resource Browsing and Discovery

Once logged in, buyers land on `HomeContent`. Inside `HomeScreen`:

- Dynamic sliders: Auto-slide banners display promotional messages or announcements.
- Category browsing: Horizontal lists categorized by resource types. Tapping a category (e.g. `'Notes'`) filters the displayed products.
- Product Cards listing: Renders all active items uploaded by sellers, excluding sold items, using `GridView.builder`.
- Search Screen routing: Tapping the Search bar opens a filtered search page using `.contains()` on product titles.
- Product Details Screen: Selecting an item displays the product's details: title, condition, price, description, seller's name, rating, reviews count, and stock quantity.

---

3. wishlist (Saved Items)

- Favorites Toggle: TAPPING the favorite heart icon in the listing card or details screen calls:

`await AppState.toggleFavorite(product);`

- Firestore Operations:
- If the favorite document (`${uid}_${productId}`) already exists in the `savedItems` collection, it is deleted.
- If it does not exist, a new document is written:

`await _firestore.collection('savedItems').doc("${uid}_${product.id}").set({ 'userId': uid, 'productId': product.id, ... });`

- UI Syncing: A Firestore stream listener watches changes in `savedItems` filters by `userId == uid`, and updates `wishlistNotifier`, immediately updating the wishlist screen and heart indicators.

---

4. shopping Cart and Checkout Lifecycle

- Add to Cart: Tapping "Add to Cart" on the details screen triggers:

`await AppState.addToCart(product);`

- Checks if the document `${uid}_${productId}` exists in the `cart` collection.
- If yes, it updates the quantity: `update({'quantity': currentQty + 1})`.
- If no, it creates a new document with sharded metadata.

- Cart Management: In `CartScreen`, buyers can increase/decrease quantities, triggering `incrementCartItem` and `decrementCartItem` updates in Firestore.
- Checkout & Order Placement: When the buyer clicks "Checkout", the app executes:

`await AppState.checkout();`

This uses a Firestore batch transaction:

- For each cart item, it writes a new document to the `orders` collection with status `'Pending'`.
- Deletes the item from the `cart` collection using `batch.delete()`.
- Decrements product stock field using `batch.update()`.
- Triggers notifications: "Order Placed" notification to the buyer, and "New Order" notifications to the sellers.
- Commits the batch transaction: `await batch.commit();`.

---

5. Order Tracking & Communications

- Order lists: Buyers access their orders under "My Orders" in the Profile screen via `orders_screen.dart`.
- Tabs: Separates orders into "Active" and "Completed" based on status.
- Real-time updates: The screen listens to the `orders` collection, matching documents where `buyerId == uid`. Any status update performed by the seller updates the buyer's UI instantly.
- Negotiations: Buyers negotiate directly with sellers by tapping "Chat with Seller" on details pages, launching real-time conversations.

PART 8: CHAT SYSTEM & MESSAGING FLOW

The peer-to-peer real-time communication system in StudySwap is built on top of Firestore document reference listeners.

1. Data Schema Hierarchy in Firestore

To minimize document size and optimize cost queries, StudySwap organizes discussions into two hierarchies:

- Conversations Collection: The root folder `conversations` contains summary channels. Each document represents a chat channel. E.g.

`conversations/docId`

This document stores participants, buyer/seller UIDs, product names, sharded thumbnails, last sent preview, unread counters, and custom sorting metadata.

- Messages Collection (Subcollection): Under each conversation document, there is a nested subcollection folder named `messages`. E.g.

`conversations/docId/messages/messageId`

This collection stores the individual text documents, containing `senderId`, `receiverId`, `text`, `isRead`, and `timestamp` fields.

---

2. Dynamic Real-time Chat Streaming

Chat messages load dynamically when a user opens the chat dialog:

1. Active Message Listener: When `chat_screen.dart` loads, it triggers:

`AppState.listenToMessages(conversationId);`

2. query subscription: This subscribes to the messages subcollection sorted by timestamp:

`_firestore.collection('conversations').doc(conversationId).collection('messages').orderBy('timestamp', descending: false).snapshots().listen((snapshots) { ... })`

3. state update: As snapshots arrive, they are mapped to `ChatMessage.fromFirestore()` objects and appended to `conv.messagesNotifier.value`.
4. UI Rebuilds: The screen displays the messages list via a `ValueListenableBuilder` listening to `messagesNotifier`, which automatically rebuilds when the list is populated.
5. auto-read Triggers: The stream listener automatically resets the unread count for the current user:

`_resetUnreadCount(conversationId, uid);`

This updates `unreadCount.currentUserId: 0` in the parent conversation document.

---

3. Creating conversations (startOrGetConversation)

When a buyer clicks "Chat with Seller" on the product detail page, the app calls:
`AppState.startOrGetConversation(product);`

1. Duplicate prevention check: Queries the `conversations` collection to see if a channel already exists for the

matching buyer, seller, and product:
`_firestore.collection('conversations').where('buyerId', isEqualTo: uid).where('sellerId', isEqualTo: sellerId).where('productId', isEqualTo: product.id).limit(1).get()`

2. Redirect existing channel: If a document is returned, it returns that document's ID.
3. Construct new channel: If no document is found, it creates a new conversation document:
`_firestore.collection('conversations').doc()`

This document stores sharded information (buyer/seller details, listing image metadata) so the chat inbox can render list items without performing multiple subqueries.

---

4. Sending Messages (addMessage)

When a user types a message and clicks the send button:
`AppState.addMessage(convId, text);`
This transaction uses a Firestore batch write:

1. Append Message document: Creates a new document in the messages subcollection:
`batch.set(messageRef, { ... 'senderId': uid, 'text': text, 'timestamp': FieldValue.serverTimestamp() });`
2. Update Parent Conversation: Updates the parent document to refresh the last message preview, set a new server timestamp, and increment the recipient's unread counter:
`batch.update(parentRef, { 'lastMessage': text, 'lastMessageTime': serverTimestamp(), 'unreadCount.$recipientId': FieldValue.increment(1) });`
3. Commit Batch: Writes all changes atomicly.
4. Alerts trigger: Automatically creates a notification document:
`addNotification("New message from Seller/Buyer", text, type: "Messages", userId: recipientId, relatedId: convId)`
This triggers a real-time message notification on the recipient's device.

PART 9: FIREBASE STORAGE & IMAGE HANDLING

StudySwap handles binary files (e.g., student listing photos and profile pictures) using Firebase Storage.

1. Local Image Selection (FilePicker)

When a user updates their profile picture or uploads a new listing:

- Pick selection: Rather than standard camera libraries, the app calls `FilePicker.pickFiles()` to select a file from the device storage.
- File filters: The picker restricts file types to `['jpg', 'jpeg', 'png', 'webp']` and checks the selected file size, rejecting files larger than 5MB to optimize network performance.

---

2. Upload Flow to Firebase Storage

- Path structure: Images are stored under organized folder structures.
- Listing images: `product_images/$productId`
- Profile images: `profile_images/$uid`
- Platform-Specific Streams:
- On Mobile (Android/iOS): Uploaded directly from the local path:

`UploadTask uploadTask = storageRef.putFile(File(localPath));`

- On Web: Local file paths are not accessible for security reasons. Instead, the raw image bytes are uploaded as a data URI:

`UploadTask uploadTask = storageRef.putData(bytes, SettableMetadata(contentType: 'image/jpeg'));`

- Await completion: The app sits on an `await uploadTask` hook to monitor upload progress.

---

3. Obtaining Download URLs

Once the upload finishes successfully:

- Public URL mapping: The app requests the public download URL:

`final downloadUrl = await snapshot.ref.getDownloadURL();`

- URL storage rationale: The download URL is a secure tokenized HTTPS link pointing to the file stored in Google Cloud Storage.
- Database writing: The retrieved URL is stored in the Firestore document under the `imageUrl` or `profileImage` fields.

---

4. Why We Store URLs Instead of Raw Image Bytes

- Firestore Size Caps: Firestore documents are capped at a maximum size of 1MB. A single raw image can easily exceed this limit.

- Cost and performance optimization: Firestore is charged by the number of read/write operations. Querying megabytes of raw image data on every document load would cause high latency and increase database costs.
- Caching benefits: Storing static HTTPS URLs allows the Flutter client to cache downloaded images locally using standard network image widgets, improving app responsiveness.

PART 10: REAL-TIME DATA ARCHITECTURE

StudySwap leverages Firestore's real-time capabilities to update the UI instantly without manual refreshes.

1. What is a Stream?

A Stream is a source of asynchronous data events. Instead of executing a one-time request and waiting for a response, a Stream keeps the connection open and pushes new data events whenever a change occurs.

---

2. snapshots() vs get()

- `get()`: Requests a single resource state once. It pulls data from the server (or cache), returns the results, and closes the connection.
- `snapshots()`: Subscribes to a stream of query results. The server registers a listener on the matching data path and pushes a new query snapshot to the client device whenever a document is added, updated, or deleted.

---

3. Realtime Listener Hookup in app_state.dart

During application initialization, `AppState` subscribes to real-time streams for all core collections:

```
_firestore.collection('products').snapshots().listen((prodSnapshot) {
  final List<Product> list = prodSnapshot.docs.map((doc) {
    return Product.fromFirestore(doc.data(), doc.id);
  }).toList();
  productsNotifier.value = list;
});
```

Every time a seller adds a product or a buyer purchases an item, Firestore pushes a new `prodSnapshot`. The listener maps this snapshot to a list of `Product` objects and updates `productsNotifier.value`.

---

4. UI Rebuilds: ValueListenableBuilder vs StreamBuilder

- `StreamBuilder`: Rebuilds its widget subtree whenever a stream emits a new event, but manages the subscription lifecycle directly within the widget tree.
- `ValueListenableBuilder`: Listens to a `ValueNotifier` (which stores value changes inside our state manager).
- Rationale: StudySwap uses `ValueListenableBuilder` tied to global `AppState` notifiers. This decouples our business logic from the UI layer. Only one global Firestore subscription is kept open in the background, rather than opening separate subscriptions on every screen, which optimizes performance and reduces Firestore read costs.
- Rebuilding Mechanism:

1. Firestore data changes on the server.

2. The background Stream listener in `AppState` is triggered.
3. The listener updates the `ValueNotifier` value in memory.
4. The `ValueListenableBuilder` detects the notifier change and rebuilds the corresponding widget subtree.

PART 11: STATE MANAGEMENT & APPSTATE LIFE

StudySwap organizes its business logic, database queries, and notifications into a single centralized state manager: `AppState.dart` (`lib/state/app_state.dart`).

1. Why AppState instead of Provider or Bloc?

- Simplicity and Readability: Using custom `ValueNotifiers` inside a vanilla class maps directly to native Flutter APIs without third-party dependencies.
- Single Source of Truth: Allows centralized management of authentication states, visual assets, shopping queues, and customer orders.
- Optimized performance: We subscribe to Firestore listeners once when the user authenticates, and cancel them when they log out. This prevents memory leaks and unnecessary database reads.

---

2. In-Memory Notifiers (State Holders)

`AppState` defines and stores states inside `ValueListenable` objects:

- `userRoleNotifier` (ValueNotifier<String>): Holds the current user role (`'Buyer'` or `'Seller'`).
- `productsNotifier` (ValueNotifier<List<Product>>): Active catalog data list.
- `cartNotifier` (ValueNotifier<List<CartItem>>): Cart items list.
- `wishlistNotifier` (ValueNotifier<List<Product>>): Saved items list.
- `buyerOrdersNotifier` / `sellerReceivedOrdersNotifier` (ValueNotifier<List<OrderModel>>): Respective purchase order logs.
- `conversationsNotifier` (ValueNotifier<List<Conversation>>): Chat channels list.
- `notificationsNotifier` (ValueNotifier<List<AppNotification>>): Active user alerts.
- `numNotificationsNotifier` (ValueNotifier<int>): Number of unread notifications.

---

3. Subscription Management Lifecycle

The lifecycle of state subscriptions follows these transitions:

1. Initialization (`AppState.init()`):

- Listens to authentication state changes:

`_auth.authStateChanges().listen((User? user) { ... })`

- If a user session is active, it coordinates subscriptions for their `uid` using `_setupAllSubscriptions(user.uid)`.
- If no user is logged in, it calls `_clearAllSubscriptions()` to wipe the local state cache.

2. Connecting Subscriptions (`_setupAllSubscriptions(uid)`):

- Opens persistent stream listeners for all collections, updating the corresponding `ValueNotifiers` with the received data:
- User metadata listener.

- Active products list listener.
- Saved items list listener.
- Cart items list listener.
- Buyer/Seller orders listener.
- Conversations inbox listener.
- Notifications inboxes listener.

3. Disconnecting Subscriptions (`_clearAllSubscriptions()`):

- Closes and destroys all active stream subscriptions using `.cancel()`:

`_userSubscription?.cancel(); _productsSubscription?.cancel(); ...`

- Resets all local list parameters to empty state.
- Clears memory cache.

PART 12: KEY FLUTTER WIDGETS USED IN STUDYS

StudySwap matches a clean layout design using Material Design 3 widgets. Below is the structural guide of the widgets used.

1. Structure and App-shell Widgets

- `MaterialApp`: The root widget of the application. It initializes the overall theme configurations (`lightTheme` and `darkTheme`), text styling, and custom navigation configurations.
- `Scaffold`: Implements the basic Material Design visual layout structure, providing helper parameters like `appBar`, `body`, `drawer`, `bottomNavigationBar`, and `floatingActionButton`.
- `AppBar`: Provides the top navigation banner of the app. It holds category headings, custom back navigation buttons, and search forms.
- `Drawer`: A side navigation panel that slides out from the left edge of the Scaffold. Used in `HomeScreen` to house Seller dashboards and configuration profiles.
- `BottomNavigationBar`: Used in Buyer's HomeScreen context. Holds distinct page directories (Home, Saved, Basket, Messages, and Profile), switching screen index states in-memory.

---

2. Layout and Component Position Widgets

- `Container`: A multi-purpose box model widget that groups decoration properties (gradients, borders, border radius, background color, card shadows) and layout constraints (padding, margin, width, height).
- `Card`: A widget with rounded corners and elevation shadows that separates sections like products, dashboards, and orders.
- `Stack` and `Positioned`: A stack overlays widgets on top of each other. `Positioned` locates a child relative to the edges of the stack. We use this to place "Unread Notification" counters on top of mail icons.
- `Expanded` and `Flexible`: Adjusts widget sizing inside rows or columns. `Expanded` forces a child to fill the entire remaining horizontal or vertical space. `Flexible` allows its child to size itself dynamically up to the remaining space.

---

3. Lists and Grids Widgets

- `ListView.builder`: Renders a linear directory of scrolling components. Uses a builder function to construct elements on-demand, which optimizes memory by recycling out-of-screen widgets. Used for displaying notifications, orders, or message lists.
- `GridView.builder`: Renders a two-dimensional grid of scrolling cards (e.g. products listing catalog grids), optimizing memory consumption similarly to `ListView.builder`.

---

4. Input and Forms widgets

- `Form`: A container for grouping multiple input fields. It provides a `GlobalKey<FormState>` parameter to validate all input fields at once.
- `TextFormField`: Subclass of `TextField` designed for Forms. It supports custom styling, hint labels, prefix selectors, and validator callbacks to display real-time error bubbles (e.g. email checks, empty validations).

---

5. Async Rendering and Stream Widgets

- `StreamBuilder`: Rebuilds itself on every stream emission. (Used primarily in chat message flows).
- `FutureBuilder`: Rebuilds widgets relative to a Future's execution states (e.g. ConnectionState.waiting, ConnectionState.done), allowing loaders to render while fetching profiles.

---

6. Feedback, Gestures, Visuals Widgets

- `GestureDetector`: Detects user gestures (taps, double-taps, drags, long-presses) on non-interactive parent layers like container areas.
- `CircularProgressIndicator`: A progress spinner that blocks other interactions during database transactions (login, signup, checkout, asset publishing).
- `CircleAvatar`: Displays a circular avatar container for portraits. Used to display user profile pictures.
- `Image.network` and `Image.asset`: `Image.asset` displays packaged local images. `Image.network` displays seller listing upload images fetched from Storage download URLs.
- `Hero`: Provides hero slide animations between screens (such as transitioning a product's card image of one screen into the large detail photo of the ProductDetails screen).
- `ValueListenableBuilder`: Rebuilds parts of the widget tree when the value of a `ValueNotifier` changes, optimizing performance.
- `ChoiceChip`: Renders chip selectors for categories or item conditions like "New" and "Good".

PART 13: IMPORTANT FUNCTIONS DIAGNOSIS

This chapter displays the execution flow, code fragments, parameters, and design structures of the 5 most critical functions of StudySwap.

1. checkout()

- File location: `lib/state/app_state.dart`
- parameters: None.
- Return Type: `Future<void>`
- Exact Code Snippet:

```
static Future<void> checkout() async {
  final uid = _auth.currentUser?.uid;
  if (uid == null) return;
  final items = List<CartItem>.from(cartNotifier.value);
  if (items.isEmpty) return;

  final batch = _firestore.batch();
  for (var item in items) {
    // 1. Create order
    final orderRef = _firestore.collection('orders').doc();
    batch.set(orderRef, {
      'buyerId': uid,
      'buyerName': nameNotifier.value,
      'sellerId': item.product.sellerId,
      'sellerName': item.product.sellerName,
      'productId': item.product.id,
      'price': item.product.price,
      'quantity': item.quantity,
      'status': 'Pending',
      'purchaseDate': FieldValue.serverTimestamp(),
      'createdAt': FieldValue.serverTimestamp(),
      'product': {
        'id': item.product.id,
        'title': item.product.title,
        'imageUrl': item.product.imageUrl,
        'category': item.product.category,
        'condition': item.product.condition,
        'description': item.product.description,
      }
    });

    // 2. Decrement stock
    final prodRef = _firestore.collection('products').doc(item.product.id);
    final currentStock = item.product.stock;
    final newStock = (currentStock - item.quantity).clamp(0, 999999);
    batch.update(prodRef, {
      'stock': newStock,
      'isSold': newStock == 0,
    });
```

```
    // 3. Delete from cart
    final cartRef = _firestore.collection('cart').doc("${uid}_${item.product.id}");
    batch.delete(cartRef);

    // 4. Send seller alert
    final sellAlertRef = _firestore.collection('notifications').doc();
    batch.set(sellAlertRef, {
      'notificationId': sellAlertRef.id,
      'userId': item.product.sellerId,
      'title': 'New Order Received!',
      'body': '${nameNotifier.value} bought ${item.quantity}x "${item.product.titl...
      'type': 'Sales',
      'relatedId': orderRef.id,
      'isRead': false,
      'timestamp': FieldValue.serverTimestamp(),
    });
  }

  // Send buyer alert
  final buyAlertRef = _firestore.collection('notifications').doc();
  batch.set(buyAlertRef, {
    'notificationId': buyAlertRef.id,
    'userId': uid,
    'title': 'Checkout Successful!',
    'body': 'Your order for ${items.length} items has been placed.',
    'type': 'System',
    'relatedId': '',
    'isRead': false,
    'timestamp': FieldValue.serverTimestamp(),
  });

  await batch.commit();
}
```

- Execution Flow:

1. Retrieve active Buyer user ID reference.
2. Extract cached elements from `cartNotifier`.
3. Instantiate a Firestore `WriteBatch`.
4. Loop through cart items:

- Log a new unique document in the `orders` collection.
- Calculate the remaining stock (`current - quantity`) and update the product document. If stock falls to 0, mark the product as sold.
- Delete the corresponding item document in the `cart` collection.
- Add a real-time order alert for the seller.

5. Add a checkout success alert for the buyer.
6. Commit all operations atomically via `batch.commit()`.

---

2. startOrGetConversation()

- File location: `lib/state/app_state.dart`

- parameters: `Product product`
- Return Type: `Future<String>` (returns conversation ID string)
- Exact Code Snippet:

```
static Future<String> startOrGetConversation(Product product) async {
  final uid = _auth.currentUser?.uid;
  if (uid == null) throw Exception("User not authenticated");

  // Check if conversation already exists
  final query = await _firestore.collection('conversations')
      .where('buyerId', isEqualTo: uid)
      .where('sellerId', isEqualTo: product.sellerId)
      .where('productId', isEqualTo: product.id)
      .limit(1)
      .get();

  if (query.docs.isNotEmpty) {
    return query.docs.first.id;
  }

  // Create new conversation
  final docRef = _firestore.collection('conversations').doc();
  await docRef.set({
    'participants': [uid, product.sellerId],
    'buyerId': uid,
    'sellerId': product.sellerId,
    'buyerName': nameNotifier.value,
    'sellerName': product.sellerName,
    'productId': product.id,
    'productTitle': product.title,
    'productImageUrl': product.imageUrl,
    'lastMessage': '',
    'lastMessageTime': FieldValue.serverTimestamp(),
    'unreadCount': {
      uid: 0,
      product.sellerId: 0,
    }
  });

  return docRef.id;
}
```

- Execution Flow:

1. Verify buyer session details.
2. Execute a Firestore query filtering by buyer ID, seller ID, and product ID.
3. If a matching document is found, return its document ID to navigate directly to the chat.
4. If no document exists, instantiate a new document reference with `_firestore.collection('conversations').doc()`.
5. Set the new conversation metadata and return its document ID.

---

3. seedInitialProductsIfEmpty()

- File Location: `lib/state/app_state.dart`

- parameters: None
- Return Type: `Future<void>`
- Exact Code Snippet:

```
static Future<void> seedInitialProductsIfEmpty() async {
  final snap = await _firestore.collection('products').limit(1).get();
  if (snap.docs.isNotEmpty) return; // already has products

  final list = [
    Product(
      id: "1",
      title: "Engineering Electromagnetics",
      price: 45.0,
      imageUrl: "assets/images/book_cover.jpg",
      sellerName: "John Doe",
      sellerId: "mock_seller_john",
      description: "Standard course book for BEE. Clean pages.",
      condition: "Good",
      category: "Books",
      stock: 5,
    ),
    // ... other mock products
  ];

  final batch = _firestore.batch();
  for (var p in list) {
    final ref = _firestore.collection('products').doc(p.id);
    batch.set(ref, p.toMap());
  }
  await batch.commit();
}
```

- Purpose: Seeds the database with default listing data if the collection is empty.
- Execution Flow:
1. Requests a single document from the products catalog.
2. If a document is fetched, return early.
3. If empty, loop through the mock listings array and write them to a batch.
4. Write the default catalog to Firestore using `batch.commit()`.

---

4. uploadAndUpdateProfileImage()

- File Location: `lib/state/app_state.dart`
- parameters: `String localPath`
- Return Type: `Future<void>`
- Exact Code Snippet:

```
static Future<void> uploadAndUpdateProfileImage(String localPath) async {
  final uid = _auth.currentUser?.uid;
  if (uid == null) return;

  final storageRef = FirebaseStorage.instance.ref().child('profile_images/$uid');
  String downloadUrl;
```

```
  if (kIsWeb || localPath.startsWith('data:image')) {
    final dataUri = Uri.parse(localPath);
    final bytes = dataUri.data!.contentAsBytes();
    final mime = dataUri.data!.mimeType;
    final uploadTask = storageRef.putData(bytes, SettableMetadata(contentType: mim...
    final snapshot = await uploadTask;
    downloadUrl = await snapshot.ref.getDownloadURL();
  } else {
    final file = File(localPath);
    final uploadTask = storageRef.putFile(file);
    final snapshot = await uploadTask;
    downloadUrl = await snapshot.ref.getDownloadURL();
  }

  await _firestore.collection('users').doc(uid).update({
    'profileImage': downloadUrl,
  });
}
```

- Execution Flow:

1. Verify user authentication.
2. Instantiate a Firebase Storage path reference: `profile_images/$uid`.
3. Check the client environment:

- On Web/DataURI: Extract bytes and call `putData()`.
- On Mobile: Create a File object and call `putFile()`.

4. Await file upload and fetch the public download URL.
5. Update the `profileImage` field in the user's Firestore document.

---

5. toggleFavorite()

- File Location: `lib/state/app_state.dart`
- parameters: `Product product`
- Return Type: `Future<void>`
- Exact Code Snippet:

```
static Future<void> toggleFavorite(Product product) async {
  final uid = _auth.currentUser?.uid;
  if (uid == null) return;

  final ref = _firestore.collection('savedItems').doc("${uid}_${product.id}");
  final doc = await ref.get();

  if (doc.exists) {
    await ref.delete();
  } else {
    await ref.set({
      'userId': uid,
      'productId': product.id,
      'savedAt': FieldValue.serverTimestamp(),
    });
  }
}
```

- Execution Flow:

1. Retrieve currently active user credentials.
2. Point document reference to `savedItems/${uid}_${productId}`.
3. Fetch the document using `get()`.
4. If the document exists, remove it from the wishlist using `delete()`.
5. If the document does not exist, add it to the wishlist using `set()`.

PART 14: POSSIBLE VIVA QUESTIONS & COMPLETE

Here is the breakdown of 85 targeted viva questions with answers based on StudySwap.

1. Firebase & General Backend Integration

1. Q: What backend services did you use inside StudySwap?

A: Firebase Authentication, Cloud Firestore (database), and Cloud Firebase Storage.

2. Q: Why did you choose Firebase over MySQL or MongoDB?

A: Firebase provides real-time web-sockets support (snapshots) out-of-the-box, requires no server hosting, and integrates with Flutter.

3. Q: How does Flutter connect with Firestore?

A: Via the official packages `firebase_core` and `cloud_firestore`. Initialization happens using `Firebase.initializeApp()` in `main.dart`.

4. Q: How does the application identify which user is currently logged in?

A: Using `FirebaseAuth.instance.currentUser`, which returns a user object containing their unique `uid`.

5. Q: Where do you store the user's role (Buyer/Seller)?

A: In a document under the `users` collection in Cloud Firestore, using the user's `uid` as the document ID.

6. Q: How do you protect database operations?

A: Users must authenticate. The app enforces role checks (Buyer vs. Seller) in state controllers and applies Firestore Security Rules on the backend.

7. Q: What is the difference between Document Database and Relational Database?

A: Document databases store data as JSON-like documents (NoSQL), whereas relational databases store data in tables with rows and columns (SQL).

8. Q: How does Firestore organize data?

A: Into Collections (folders) containing Documents (JSON files), which can contain nested subcollections.

9. Q: Is Firestore a SQL or NoSQL database?

A: NoSQL. It stores collections of documents rather than tables with rows and columns.

10. Q: What is the maximum size allowed for a single Firestore document?

A: 1 Megabyte (1MB).

11. Q: Why did you not store raw images directly inside Firestore?

A: Firestore documents are capped at 1MB. Instead, we upload images to Firebase Storage and save their HTTPS download URLs in Firestore.

12. Q: What is a Firestore Batch?

A: A collection of write operations (set, update, delete) that executes atomically. If one fails, the entire batch rolls back.

13. Q: Why did you use a batch write for checkout in StudySwap?

A: Checkout requires multiple updates: creating orders, decrementing stock, and clearing the cart. Using a batch ensures atomic consistency.

14. Q: What happens if checkout fails mid-way?

A: Since we use a write batch, no changes are committed to the database, preventing inconsistent states.

15. Q: What is the difference between set() and update()?

A: `set()` overwrites the entire document (or creates a new one). `update()` modifies only the specified fields of an

existing document.

16. Q: What happens when trying to update() a document that does not exist?

A: Firestore throws a FirebaseException.

17. Q: How does your app generate unique document IDs?

A: Calling `.doc()` without arguments on a collection reference generates a unique 20-character alphanumeric ID client-side.

18. Q: What type of query filters does Firestore support?

A: Range and equality operators using `.where()`, sorting using `.orderBy()`, and limiters using `.limit()`.

2. Authentication and Role Flows

19. Q: How do you handle account creation in Firebase?

A: We call `createUserWithEmailAndPassword(email, password)` and then write the user's metadata to Firestore.

20. Q: How is client validation configured for registration?

A: We use a Flutter `Form` with a `GlobalKey<FormState>`. Each `TextFormField` has validation logic for formats, lengths, and matches.

21. Q: How do you handle passwords securely?

A: Firebase Authentication manages encryption and storage on the server side. Passwords are never stored in raw text in our database.

22. Q: How does a user change their password?

A: We call `await FirebaseAuth.instance.currentUser?.updatePassword(newPassword)`.

23. Q: What is password reset flow?

A: We call `await FirebaseAuth.instance.sendPasswordResetEmail(email: email)`.

24. Q: How is a Buyer's home navigation page different from a Seller's?

A: Both share the browse page, but Sellers have a "Dashboard" option in the side drawer to access the Seller Hub.

25. Q: How does role-based routing work during app launch?

A: `AppState` retrieves the authenticated user's profile document from Firestore, reads the `accountType` field, and routes them accordingly.

26. Q: What packages did you use for picking local images?

A: `file_picker`.

27. Q: Why did you use file_picker instead of image_picker?

A: `file_picker` works reliably across both mobile and web without configuration issues.

3. Realtime Messaging & Chat Screen

28. Q: How are conversations structured in Firestore?

A: In a root `conversations` collection, where each document has a nested `messages` subcollection.

29. Q: How is a conversation uniquely identified between a buyer and seller?

A: We query the `conversations` collection for a document matching the buyer ID, seller ID, and product ID.

30. Q: How does real-time chat update the UI?

A: The chat screen maps the stream of snapshots from the `messages` collection to a list of messages.

31. Q: Why does the chat listing show unread badges?

A: Each conversation document stores an `unreadCount` map. When a message is sent, the recipient's counter is incremented.

32. Q: How is the unread badge cleared when a chat is opened?

A: We reset the current user's entry in the parent conversation's `unreadCount` map to 0.

33. Q: Why did you store product metadata in conversation documents?

A: To display item thumbnails and titles in the chat list without executing additional database reads for each conversation.

4. Seller Operations, Dashboard, & Custom Painters

34. Q: How does a Seller upload a product?

A: By filling out the form in `SellScreen` and uploading the product image to Firebase Storage, then saving the metadata and URL in Firestore.

35. Q: How does the Seller Dashboard fetch metrics?

A: It listens to the `orders` collection in real-time, filtering by `sellerId` and calculating metrics dynamically.

36. Q: How is Total Revenue calculated on the dashboard?

A: By summing `price * quantity` for all orders with a status of `'Delivered'`.

37. Q: What does the Analytics screen line chart display?

A: Dynamic daily revenue trends based on delivered order completion timestamps.

38. Q: How is the trend chart rendered?

A: Using a `CustomPainter` to draw lines, grid grids, and dots on a canvas coordinate system.

39. Q: How does the app update listing status when stock hits 0?

A: During checkout, if stock drops to 0, the write batch automatically updates `isSold` to `true` and the listing status to `'inactive'`.

5. State Management & Lifecycle

40. Q: What is the state management architecture of the application?

A: Decoupled global state management using a central `AppState` class containing `ValueNotifiers`.

41. Q: Why use ValueNotifier instead of setState?

A: `setState` only triggers rebuilds within a single stateful widget. `ValueNotifier` broadcast updates globally.

42. Q: What is the purpose of ValueListenableBuilder?

A: It listens to a `ValueNotifier` and rebuilds only its child widget subtree when the notifier's value changes.

43. Q: How do you listen to user updates in real-time?

A: We listen to the user snapshot in Firestore and update the `nameNotifier`, `profileImage`, and other states immediately.

44. Q: What occurs when AppState.signOut() is executed?

A: Cancels all Firestore stream subscriptions, clears local state, and signs out of Firebase Authentication.

45. Q: Why is canceling subscriptions on logout critical?

A: Failing to cancel listeners before sign-out causes security rule errors and memory leaks.

6. Widget Tree Architecture

46. Q: What is the purpose of a Key in Flutter widgets?

A: Keys preserve widget state when they move around the widget tree.

47. Q: What is the difference between Stateful and Stateless widgets?

A: Stateless widgets are static and redraw only when arguments change. Stateful widgets manage dynamic state locally.

48. Q: When do you use StreamBuilder?

A: To listen to streams directly inside the widget tree (e.g. Chat messages subcollection stream).

49. Q: What is the difference between Expanded and Flexible?

A: `Expanded` forces a widget to fill all remaining layout space. `Flexible` allows a widget to scale up to the remaining space.

50. Q: Why is ListView.builder preferred over standard ListView?

A: `ListView.builder` creates items on-demand, recycling off-screen items to save memory.

51. Q: What is the purpose of Hero widget?

A: It provides a shared transition animation when navigating between screens.

52. Q: What package is used for local database persistence?

A: `shared_preferences` (used to persist settings like the active theme mode).

53. Q: How do you implement dark mode statically?

A: By checking the boolean state of `darkModeNotifier` in `AppTheme` and applying target styles.

54. Q: How does the application prevent duplicate product listings from overloading the database?

A: We enforce validation criteria on titles and descriptions before publishing.

55. Q: What does FieldValue.serverTimestamp() resolve to?

A: A server-side timestamp representation, ensuring consistency regardless of client device time settings.

56. Q: How do you handle error states when Firebase is offline?

A: Firestore caches data locally, allowing operations to queue offline and sync automatically when internet connectivity returns.

57. Q: How does a buyer complete a purchase?

A: By pressing "Checkout" in the Cart screen, which triggers the atomic batch transaction.

58. Q: How does a seller view active orders?

A: Under the orders tab on the Seller Dashboard screen.

59. Q: How are notifications handled when an order status changes?

A: When the seller updates the order document status, the transaction appends an alert to the `notifications` collection.

60. Q: How is a message marked as read?

A: The recipient opens the conversation doc, updating `isRead` to true for all incoming elements in the subcollection.

61. Q: What are the main tabs in screen navigation?

A: Home, Wishlist, Cart, Messages, and Profile.

62. Q: Can a seller buy their own items?

A: The UI prevents listing owners from adding their own items to the cart or checking out.

63. Q: How does user role selection work during signup?

A: By tapping interactive cards on the registration screen, which sets the `accountType` variable.

64. Q: How does the application display search results?

A: Using simple text queries against the titles in our local products list.

65. Q: How is a user's phone number validated?

A: Standard regular expression checking in form inputs (expects 11 digits format).

66. Q: What happens when an order status is marked as 'Delivered'?

A: The system logs the delivery timestamp, increments seller completed sales metrics, and updates dashboard charts.

67. Q: Why did you choose 5MB as the image upload limit?

A: To prevent network lag and optimize Firestore document loading speeds.

68. Q: How do you show loading screens during network operations?

A: By managing a boolean state (e.g., `_isPublishing = true`) and displaying a overlay modal containing a `CircularProgressIndicator`.

69. Q: How is the total number of notifications displayed?

A: Via a red badge over the notification icon, driven by a `ValueListenableBuilder` listening to `numNotificationsNotifier`.

70. Q: How do you handle file uploads on the Web?

A: By extracting image bytes from file picker outputs and uploading them as data URIs via `putData`.

71. Q: What is the purpose of MaterialApp's routing layout?

A: Maps paths to pages dynamically.

72. Q: What occurs when a product is marked as a favorite?

A: We write a document containing user and product references to the `savedItems` collection.

73. Q: How is the category list structured on the home screen?

A: A horizontal list of chip-like items containing category names and icons.

74. Q: How are recent seller transactions calculated?

A: The dashboard pulls the last 5 completed or pending transactions from the order model lists.

75. Q: How does custom painter coordinate systems render graph points?

A: Maps revenue values to relative Y offsets, and days to X offsets on a canvas container.

76. Q: Explain the difference between Stream and Future.

A: A `Future` returns a single asynchronous result once, whereas a `Stream` delivers multiple asynchronous events over time.

77. Q: What is build context in Flutter?

A: BuildContext is a reference that defines where a widget sits in the widget tree.

78. Q: What is the purpose of safe area?

A: Keeps your UI elements from being cut off by device hardware notches and screen bezels.

79. Q: How does double.tryParse help in database validation?

A: Converts text to double, returning null if formatting fails, which helps validate price inputs.

80. Q: Why is dispose method important?

A: Closes text controller instances and UI listeners to prevent memory leaks.

81. Q: How is email format validated during signup?

A: Using regular expressions (`RegExp`) to ensure standard university email formats.

82. Q: What does ScaffoldMessenger handle?

A: Displays temporary, context-aware notification banners (SnackBars) at the bottom of the screen.

83. Q: How do you update listing stock?

A: In `MyListingsScreen`, tapping on an item opens `EditListingScreen` to submit changes.

84. Q: Can buyers see sold items in search listings?

A: The product stream filters out products marked as sold (`isSold == true`) so they are hidden from search lists.

85. Q: What is hot reload in Flutter?

A: Compiles updated code and updates the widget tree directly without restarting the application engine.

PART 15: TEACHER-LEVEL VIVA QUESTIONS & ANS

Here is the compilation of 18 complex, high-difficulty concepts and scenario questions.

1. Cloud Firestore Internal Operations

1. Q: How does Firestore maintain database changes across offline clients?

A: It uses an offline caching layer. Writes are stored in an offline queue locally. When the connection is restored, the client uploads the queue to the cloud database.

2. Q: Why did you choose snapshots() instead of periodic polling?

A: Polling creates high database read overhead and consumes battery life on client devices. Snapshots hook up persistent socket connections and stream changes instantly.

3. Q: What is a Firestore Index, and did you define any custom indexes?

A: Indexes allow Firestore to query large collections efficiently. Yes, custom composite indexes were defined for the chat system queries that filter by multiple fields: `buyerId == uid` + `sellerId == sellerId` + `productId == productId` sorted by `lastMessageTime`.

2. Transactions and Write Operations

4. Q: What is the difference between dry-running/writing a Batch vs using a Firestore Transaction?

A: Batches execute several write operations atomically without reading first. Transactions allow both reads and writes. If a document changes during a transaction, the operation retries automatically.

5. Q: Why is checkout() implemented as a Batch instead of a Transaction?

A: The required product stock quantity is already cached in `cartNotifier` in-memory. We just need to commit writes (creating orders, updating stock, deleting cart items) atomically, which is what a Batch is optimized for.

6. Q: What are the latency impacts of Firestore Batch writes?

A: Committing a batch executes in a single round-trip database write, reducing network latency compared to execution of separate writes.

3. Data Structure and Sharding

7. Q: Why is storing all messages in a single array inside the conversation document bad?

A: A single Firestore document is capped at 1MB. A long conversation would quickly exceed this limit. Storing messages in a subcollection allows infinite growth.

8. Q: Why did you duplicate data (e.g. sellerName or productTitle) in conversation and cart documents?

A: In NoSQL, this is called denormalization. We duplicate fields to render listing cards or message details immediately, avoiding the need for multiple document reads.

9. Q: How do you prevent data inconsistency when a seller edits a product title after an order is placed?

A: During checkout, we copy a snapshot of the product details at that moment into the order document. This preserves the purchase details regardless of future catalog changes.

4. State Subscriptions and Memory Optimizations

10. Q: Why is extending ChangeNotifiers or ValueNotifiers better than using Global State context directly for all views?

A: It decouples core business logic from widget builds. App state lives in a clean, stateful Dart class rather than inside the UI widgets.

11. Q: How do you prevent memory leaks when managing multiple stream subscriptions?

A: We store listener references in `StreamSubscription` properties and call `.cancel()` on them inside the `signOut()` method.

12. Q: When does Flutter rebuild a ValueListenableBuilder?

A: When the `ValueNotifier` changes its internal value pointer (for objects, `value = updatedObj` must be assigned to trigger rebuilds).

5. Security & Validation Rules

13. Q: How do you prevent a user from claiming to be someone else in chat messages?

A: By validating inputs. Additionally, Firestore security rules ensure that `request.auth.uid == data.senderId`.

14. Q: How did you implement real-time validation on forms?

A: We used `TextFormField` validator callbacks, which return warning strings if the input does not meet the specified format.

15. Q: What happens if a buyer attempts to enter negative numbers in the cart?

A: We validate inputs inside the cart controllers. The quantity cannot drop below 1.

6. Architecture Scenarios

16. Q: If the user drops connection during a File Upload, how does the app recover?

A: Firebase Storage's `UploadTask` monitors progress. If it fails, our `catch` block intercepts the error, dismisses the loading indicator, and displays a SnackBar warning.

17. Q: Why is `DefaultFirebaseOptions.currentPlatform` preferred over inline configurations?

A: It automatically selects correct platform keys (Android Client vs. iOS Client vs. Web) at build time, simplifying configuration.

18. Q: How would you scale StudySwap to support 100,000 active students?

A: We would apply database sharding for messaging, use Algolia/ElasticSearch for catalog searches, and implement Cloud Functions to handle notifications.

PART 16: INTERVIEW STYLE REVISION & SUMMARY

This chapter serves as a rapid overview checklist for the final minutes before the viva.

1. Quick Definitions Checklist

- Stream: An asynchronous sequence of data events (e.g. Firestore snapshots).
- Future: A computation that returns a single result asynchronously (e.g. Firebase sign-in).
- Batch: Atomically groups multiple write operations (set, update, delete) to execute in a single round-trip call.
- Transaction: A series of reads and writes that execute atomically and retry if a conflicts arise.
- Denormalization: Duplicating data (e.g. embedding product details inside orders) to speed up queries.
- ValueListenableBuilder: Rebuilds its child widget tree dynamically when a watched notifier changes value.

---

2. Key Technology Statistics

- Backend Database: Cloud Firestore (NoSQL, document-oriented).
- File Storage: Firebase Storage (binary storage, bucket architecture).
- Security: Firebase Authentication (email-based credentials management).
- State Management: Centralized `AppState.dart` using global `ValueNotifiers`.
- Platform support: Configured to support both Android and iOS devices, and Web compilation options.

---

3. Top 5 Common Student Mistakes in Viva

1. Confusion between get() and snapshots():
- Mistake: Explaining `get()` as real-time, or stating `snapshots()` is a single read.
- Correction: Remember `get()` reads once. `snapshots()` returns a real-time stream.
2. Mixing setState() and ValueNotifier:
- Mistake: Stating `ValueNotifier` forces the whole screen to redraw.
- Correction: `ValueNotifier` rebuilds only the scoped widget tree inside `ValueListenableBuilder`.
3. Inability to explain Checkout logic:
- Mistake: Not knowing that stock changes happen atomicly via database batches.
- Correction: Be ready to explain the write batch transaction in detail.
4. Not knowing where images are stored:
- Mistake: Saying listing images are saved inside Firestore collections.
- Correction: Listing images live in Firebase Storage. Only their download URLs are saved in Firestore.
5. Inability to explain how user roles are managed:
- Mistake: Stating roles are configured inside Firebase Authentication panel properties.
- Correction: Firebase Authentication only manages credentials. We save the `accountType` field in the user's Firestore document.

---

4. Final Five-Minute Viva Checklist

- Run `flutter analyze` locally to ensure there are no compilation errors.
- Verify your database connection config files are current.
- Refresh your understanding of client-side validation logic for registration.
- Review the checkout write-batch flow.
- Remember: the primary color of StudySwap is Indigo/Royal Violet.